

Versionamento de Código

#Git

O que é versionamento?

O **versionamento de código** é o processo de registrar e gerenciar diferentes versões de um projeto ao longo do desenvolvimento.

Utilizando o Git, podemos:

- Acompanhar as modificações realizadas.
- Consultar versões anteriores.
- Identificar quando e por quem a alteração foi feita.
- Trabalhar colaborativamente sem sobrescrever o trabalho dos outros.

O [Git](#) organiza isso utilizando 3 conceitos fundamentais: **Staging Area, Commits e Branches**.

1. Staging Area - `git add`

A **Staging Area** é uma área intermediária onde preparamos as alterações que desejamos incluir no próximo Commit.

O comando `git add` **não salva** uma nova versão do projeto. Ele apenas seleciona as alterações que serão registradas posteriormente.

Principais Comandos:

- `git add .` → prepara **todas** as alterações dentro do diretório atual e seus subdiretórios.
- `git add arquivo.txt` → prepara somente o arquivo selecionado.
- `git status` → verifica quais alterações estão ou não preparadas.
- `git restore --staged arquivo.txt` → retira o arquivo da preparação sem apagar suas modificações locais.

Importante: *adicionar arquivos à Staging Area é diferente de criar um commit*

2. Commits - `git commit`

Um **commit** representa um registro de uma versão do projeto em determinado momento.

Ele funciona como uma "*fotografia*" das alterações preparadas, permitindo consultar o histórico de desenvolvimento.

Para criar um **commit** utilizamos:

```
git commit -m "mensagem descritiva"
```

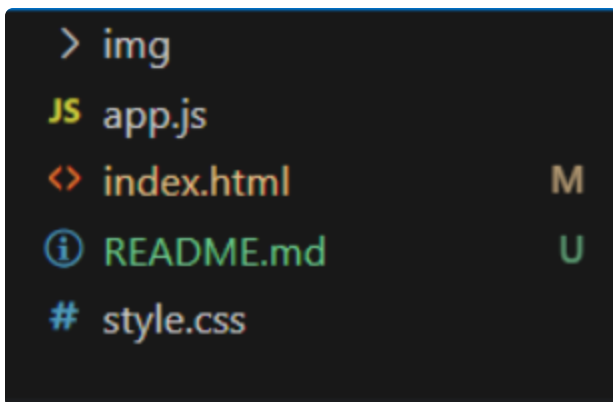
O parâmetro `-m` permite informar uma mensagem diretamente pelo terminal.

Cada commit possui:

- **Hash:** identificador único do commit.
- **Autor:** responsável pelo registro.
- **Data e Hora:** quando foi realizado.
- **Mensagem:** descrição da alteração.
- **Referências ao histórico anterior:** permite conectar as diferentes versões do projeto.

Sinalizações em arquivos do VSCode

Quando estamos trabalhando em um projeto, ao adicionar ou alterar qualquer arquivo aparece uma sinalização:



O que significa?

- **M:** *Modified* (Modificado) → O arquivo já existia no repositório e foi alterado.
- **U:** *Untracked* (Não rastreado) → O arquivo ainda não existia no repositório e ainda não teve commit feito.

Boas práticas para commits

Um commit deve representar uma alteração e apresentar uma mensagem clara sobre o que foi desenvolvido.

A convenção utilizada para **boa prática** é a **Conventional Commits**.

Prefixo	Utilização	Exemplo
feat:	Nova funcionalidade	feat: adiciona tela de login
fix:	Correção de erro	fix: corrige validação do formulário
docs:	Documentação	docs: atualiza README
refactor:	Refatoração	refactor: reorganiza componentes
chore:	Manutenção (Tarefas diversas)	chore: atualiza dependências
style:	Formatação	style: remove espaços em branco

Além desta convenção, é trabalhada também com as seguintes boas práticas:

- **Mensagem curta e concisa:** No máximo 72 caracteres.
- **Uso de verbo no infinitivo:** "adicionar", "corrigir", "atualizar".
- **Evitar detalhes técnicos:** Não incluir detalhes complexos na mensagem.

3. Branches - Ramificações

Uma **branch** é uma linha de desenvolvimento independente dentro de um [repositório](#).

Ela permite desenvolver novas funcionalidades, corrigir erros ou testar modificações sem alterar diretamente a linha principal do projeto.

A **Branch Principal** recebe o nome de `main`.

Para que utilizar branches?

Imagine que seu projeto está funcionando normalmente, mas você precisa desenvolver um sistema de autenticação.

Em vez de modificar diretamente na `main`, é possível criar uma nova branch chamada `feat/login`.

Dessa maneira, o desenvolvimento acontece separadamente. Assim, quando estiver concluído e testado, poderá ser integrada à branch principal.

Principais comandos:

Comando	Função
<code>git branch</code>	Visualiza as branches locais.

Comando	Função
<code>git switch -c feat/login</code>	Cria uma branch e muda pra ela.
<code>git switch main</code>	Retorna para a branch principal.
<code>git merge feat/login</code>	Integra a branch login à branch atual .
<code>git branch -d feat/login</code>	Exclui a branch local após sua integração.

OBS: o comando `git branch nome` também cria uma branch, mas não muda automaticamente para ela.

O que é Merge?

O merge é o processo de integrar o histórico de uma branch ao de outra.

Por exemplo, para integrar uma funcionalidade à branch principal:

1. `git switch main` → acessar a branch principal.
2. `git merge feat/login` → integrar as alterações da outra branch na `main`.

Caso duas branches tenham modificado trechos incompatíveis de um mesmo arquivo, poderá ocorrer um **conflito de merge**, que precisará ser resolvido manualmente antes de concluir a integração.

4. Funcionamento em Conjunto

O fluxo segue uma sequência lógica:

Diretório Trabalhado → Staging Area → Repositório Local → Repositório Remoto

Etapa	Responsabilidade	Comando
Diretório Trabalhado	Local onde é editado os arquivos.	-
Staging Area	Prepara as alterações selecionadas.	<code>git add</code>
Repositório Local	Registra uma nova versão no histórico.	<code>git commit</code>
Repositório Remoto	Recebe os commits enviados (GitHub).	<code>git push</code>

As **branches** organizam diferentes linhas de desenvolvimento, cada uma com o seu próprio histórico de commits.

Exemplo de fluxo completo

Por exemplo o desenvolvimento de uma nova funcionalidade:

1. `git switch main` → acessar a branch principal.
2. `git pull origin main` → atualizar a branch `main`
3. `git switch -c feat/login` → criar uma branch para funcionalidade.
4. Desenvolvimento da aplicação...
5. `git status` → conferir as alterações.
6. `git add .` → preparar as alterações intencionais.
7. `git commit -m "feat: adiciona tela de login"` → registrar a versão.
8. `git switch main` → retornar à branch principal.
9. `git merge feat/login` → integrar a funcionalidade.
10. `git push origin main` → enviar a versão atualizada ao GitHub.